

• O FUTURO RODA EM MYSQL

MySQL BR Conf 2026



O Binlog ao Limite: Configurando e Otimizando o MySQL para CDC com Debezium e Kafka

Marcio Junior Vieira
CEO & Data Scientist, Ambiente Livre

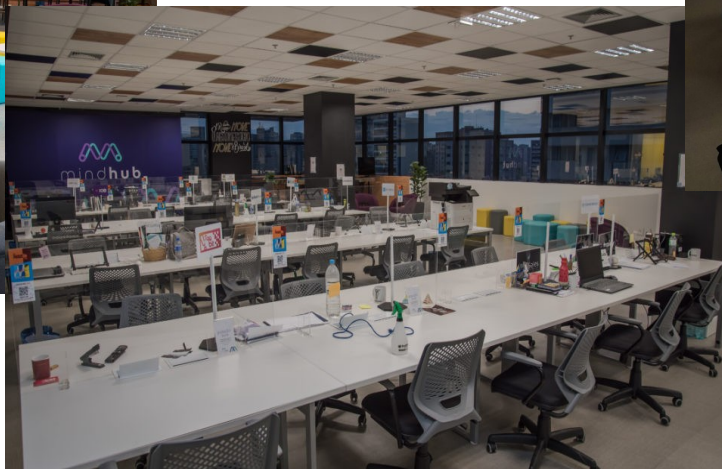
Quem Sou eu...

- **Usuário MySQL** deste 1999...
- **Data Scientist** na Ambiente Livre | 27 anos de FLOSS | 5.000+ profissionais treinados.
- Atuando como Cientista de Dados, Engenheiro de Dados e Arquiteto de Software.
- Como **professor**, lecionou nos MBAs em Big Data & Data Science, IA e BI da Universidade Positivo e no MBA de IA e Machine Learning da FIAP.
- Trabalhou como **pesquisador** pela Universidade de Brasília e pela Universidade Federal de Goiás, no CIAP, Centro de Colaboração Interinstitucional de Inteligência Artificial Aplicada às Políticas Públicas.

Sobre a Ambiente Livre

Open Software for Business

- Fundada em 2004 com foco em consultoria com software livre.
- Experts em 34 soluções para geração de negócios com Software Livre/Código Aberto.
- Atualmente estamos sediados no Hub de Inovação Mindhub em Curitiba.



Nosso Ecossistema de Serviços



Query tradicional drena o banco que deveria servir produção.

- **Polling / query tradicional.**

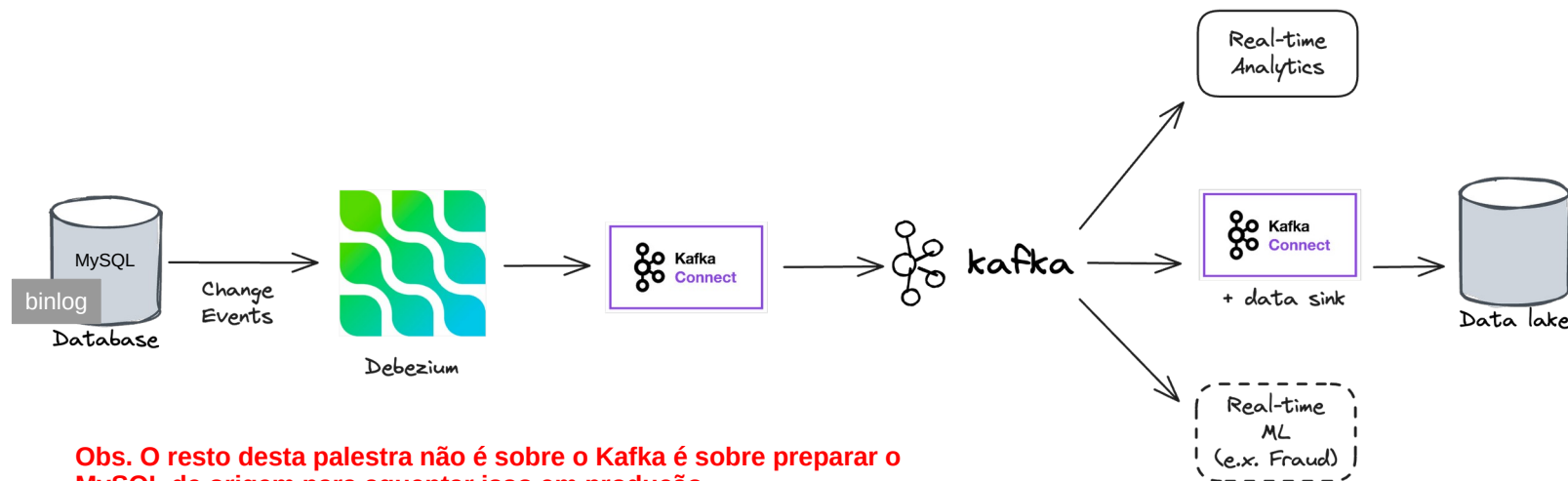
- ✗ SELECT periódico varre a tabela inteira.
- ✗ Concorrência de locks com o OLTP de produção.
- ✗ CPU e I/O consumidos por trabalho redundante.
- ✗ Latência de minutos até horas para refletir mudanças.

- **CDC / Streaming.**

- ✓ Lê o log de transações, não a tabela.
- ✓ Sem varredura, sem lock de leitura em produção.
- ✓ Consome só o que mudou, em tempo real.
- ✓ Latência de milissegundos a segundos.

CDC em uma frase: ler o log, não a tabela.

- (1) O MySQL já registra cada mudança de linha no binlog, para replicação.
- (2) CDC aproveita esse log como fonte de eventos, nada de polling.
- (3) O dado chega no Kafka como evento, pronto para Data Lake ou microserviço.



Obs. O resto desta palestra não é sobre o Kafka é sobre preparar o MySQL de origem para aguentar isso em produção.

Imagem Fonte: <https://sudohainguyen.github.io/2023-11-20-change-data-capture/>

© 2026, Ambiente Livre. Todos direitos reservados. www.ambientelivre.com.br



“O sucesso do CDC não está na ferramenta de destino. Está em como o seu MySQL de origem está preparado.”

O Debezium se registra como um servidor Slave



- 1 Handshake de replicação: registro como servidor Slave (COM_REGISTER_SLAVE)
- 2. Solicita o binlog dump a partir de uma posição (arquivo+offset) ou GTID
- 3. Lê o log de eventos linha a linha — sem tocar nas tabelas

Do my.cnf ao evento no Kafka, a mesma mudança, três formatos

```
[mysqld]  
server-id=223344  
log_bin=mysql-bin  
binlog_format=ROW  
binlog_row_image=FULL  
binlog_expire_logs_seconds=604800  
gtid_mode=ON  
enforce_gtid_consistency=ON
```

- **É daqui que tudo começa, sem isso, não existe evento pra capturar.**

binlog, via SHOW BINLOG EVENTS

- Log_name: mysql-bin.000123
Pos: 1547
Event_type: Update_rows
Server_id: 223344
End_log_pos: 1842
Info: table_id: 118 flags: STMT_END_F
- **O binlog não guarda SQL, guarda o efeito da mudança, linha a linha.**

JSON do evento

```
{
  "before": {
    "id": 4231,
    "status": "pendente",
    "valor": 199.90
  },
  "after": {
    "id": 4231,
    "status": "pago",
    "valor": 199.90
  },
  "source": {
    "db": "vendas",
    "table": "pedidos",
    "file": "mysql-bin.000123",
    "pos": 1842,
    "gtid": "3E11FA47-71CA-11E1-9E33-C80AA9429562:23",
    "ts_ms": 1732989123000
  },
  "op": "u",
  "ts_ms": 1732989123456
}
```

- **O 'antes' e o 'depois' após configurar binlog_row_image, cada campo desse JSON tem origem direta numa decisão de configuração.**

binlog_format: a decisão que quebra ou salva o CDC

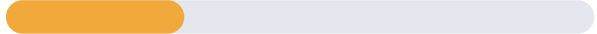
- **STATEMENT:** Grava o comando SQL
NOW(), RAND() e funções não determinísticas quebram a reprodução linha a linha
- **MIXED:** Alterna automaticamente
Ainda pode cair em STATEMENT sem aviso, imprevisível para CDC.
- **ROW:** Grava o dado, linha a linha.
Único formato determinístico, obrigatório para Debezium.

```
mysql> SHOW VARIABLES LIKE 'binlog_format';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| binlog_format | ROW   |
+-----+-----+
1 row in set (0.00 sec)
```

binlog_row_image: quanto do “antes” você quer guardar

- **FULL:** Grava todas as colunas, antes e depois
Payload completo no Kafka, mais I/O de escrita.

Peso no I/O de escrita
- **NOBLOB:** Omite BLOB/TEXT não alterados
Meio-termo, economiza em tabelas com colunas grandes

Peso no I/O de escrita
- **MINIMAL:** Só as colunas que mudaram + chave
Menor I/O, mas perde contexto de valores não alterados.

Peso no I/O de escrita

```
mysql> SHOW VARIABLES LIKE 'binlog_row_image';  
+-----+-----+  
| Variable_name | Value |  
+-----+-----+  
| binlog_row_image | FULL |  
+-----+-----+  
1 row in set (0.00 sec)
```

Retenção do binlog: o prazo de validade dos seus dados

- **binlog_expire_logs_seconds:**
Define há quanto tempo os binlogs ficam disponíveis em disco antes de serem descartados.
- **Regra prática:** a retenção precisa cobrir o pior downtime possível do pipeline , não o dia a dia normal.



O risco silencioso

Se o conector cai (manutenção, deploy, incidente) por mais tempo que a retenção, o binlog necessário já expirou.

Resultado: perda de eventos e a necessidade de um novo snapshot completo

Disco: o custo invisível de ROW + FULL

- ROW + FULL maximiza fidelidade de dados, e move o gargalo da CPU para o I/O de disco.



Separe o disco

Volume dedicado para binlog, fora do disco de dados



Monitore IOPS

Throughput de escrita é o sinal de alerta antes da CPU saturar



Dimensione com margem

Pico de tráfego de escrita do CDC deve caber, não só a média

Snapshot inicial em tabelas de terabytes.

- Antes de ler o binlog incrementalmente, o Debezium precisa capturar o estado atual da tabela.

```
SELECT * FROM THE_BIG_TABLE;
```

Snapshot ingênuo em tabela de TBs = lock prolongado + I/O storm competindo com produção

Risco de impacto em produção durante o snapshot

Três estratégias, cada uma com seu trade-off

1

Incremental Snapshot

Chunking por chave primária,
sem lock global da tabela

Mais lento, porém não bloqueia

2

Réplica dedicada

Isola o custo do snapshot
da produção

Exige infraestrutura extra

3

schema_only + backfill

Só a estrutura via CDC; dados
históricos migrados à parte

Backfill manual, mais controle

Não existe bala de prata, a escolha certa depende do tamanho da tabela e da janela de manutenção disponível.

Os limites que você não pode ultrapassar em produção

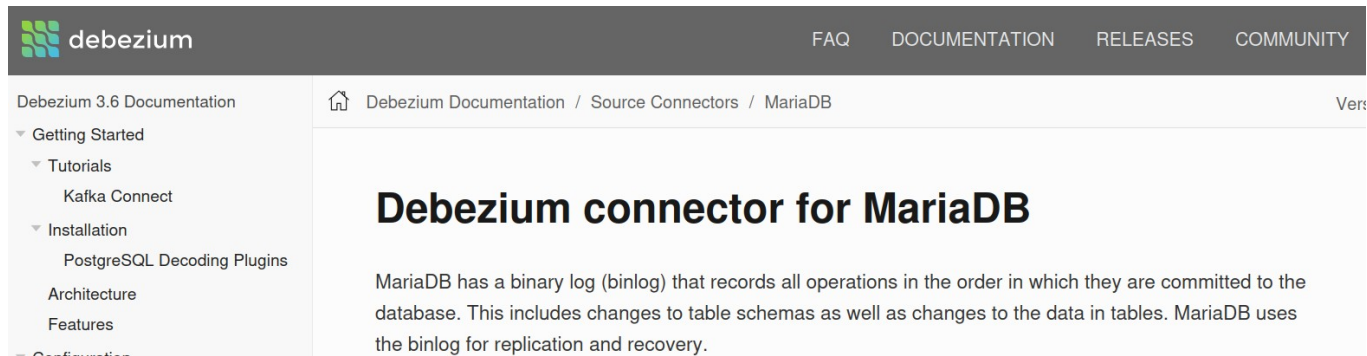
- ✔ binlog_format = ROW.
- ✔ binlog_row_image adequado ao caso de uso (FULL, NOBLOB ou MINIMAL).
- ✔ Retenção calculada pelo pior cenário de downtime, não pela média.
- ✔ Disco dimensionado para o pico de I/O de replicação.
- ✔ Estratégia de snapshot testada com volume real antes de ir para produção.



**“O sucesso do CDC começa
no MySQL/Database de
origem.”**

Compatibilidade

- Tudo que vimos, ROW, row_image, retenção, disco, snapshot, vale igual para MariaDB. **A única diferença é o GTID, que tem formato próprio e incompatível com o do MySQL, e o fato de que hoje existe um conector dedicado do Debezium para MariaDB, não mais o mesmo conector do MySQL.**



Web

- Change Data Capture (CDC) pattern: another real-time data ingestion approach <https://sudohainguyen.github.io/2023-11-20-change-data-capture> acesso em 08 de Julho de 2026.
- Ambiente Livre Labs - <https://github.com/ambientelivre/labs>

Obrigado



Marcio Junior Vieira

CEO | Data Scientist | Palestrante |
Pesquisador | Professor



Marcio Junior Vieira

marcio@ambientelivre.com.br

Redes:

@ambientelivre

@ambientelivreopensource

<https://www.linkedin.com/in/mvieira1/>

Slide da Palestra será publicada em:

Linkedin: <https://www.linkedin.com/in/mvieira1/>